Pre Test

1. Jelaskan secara singkat tujuan penggunaan klausa JOIN pada basis data relasional dan apa kelemahannya jika digunakan pada tabel dengan jutaan baris data!

Tujuan Penggunaan JOIN: Klausa JOIN digunakan untuk menggabungkan data dari dua tabel atau lebih yang saling berelasi berdasarkan kolom kunci (seperti *primary key* dan *foreign key*). Tujuannya agar basis data tetap ternormalisasi dengan baik tanpa terjadi duplikasi data yang tidak perlu.
Kelemahan utama penggunaan klausa JOIN pada tabel yang berisi jutaan baris data adalah terjadinya penurunan performa eksekusi yang drastis karena sistem manajemen basis data (*DBMS*) harus melakukan proses pencocokan baris data secara *on-the-fly*—seperti menggunakan algoritma *nested loop join* atau *hash join*—setiap kali *query* dijalankan. Selain itu, operasi ini akan menguras kapasitas sumber daya perangkat keras (CPU dan memori RAM) secara masif karena volume kombinasi data yang harus dievaluasi sangat besar. Beban komputasi server juga akan berlipat ganda jika JOIN tersebut dikombinasikan dengan fungsi agregasi yang berat (seperti SUM atau GROUP BY), sehingga membuat waktu tunggu *query* menjadi sangat lambat dan berpotensi membebani kinerja keseluruhan sistem basis data.

2. Apa yang dimaksud dengan redundansi data, dan mengapa dalam konsep "Denormalisasi" redundansi ini sengaja diciptakan?

Redundansi data adalah kondisi di mana data atau informasi yang sama disimpan secara berulang pada lebih dari satu tempat atau kolom di dalam basis data.

Dalam konsep Denormalisasi, redundansi sengaja diciptakan untuk mengoptimalkan performa baca (*read performance*). Dengan menyimpan data turunan atau hasil kalkulasi secara langsung (meskipun berulang), sistem tidak perlu lagi melakukan proses JOIN yang rumit dan fungsi agregasi yang berat setiap kali data ingin ditampilkan, sehingga eksekusi *query* menjadi jauh lebih cepat.

3. Berikan satu contoh konkret bagaimana sebuah atribut turunan dapat menghindari penggunaan fungsi agregasi (seperti SUM) dalam *query*!

menambahkan kolom atribut turunan (seperti total_belanja) langsung pada tabel master (pelanggan), hasil kalkulasi agregasi SUM disimpan secara permanen di dalam kolom tersebut. Akibatnya, saat menampilkan data, query menjadi sangat sederhana dan tidak lagi memerlukan fungsi agregasi SUM ataupun klausa JOIN

Praktikum

1. Eksplorasi JOIN (Tahap 1): Tulis dan eksekusi satu *query* yang menggunakan JOIN (minimal 2 tabel) dikombinasikan dengan fungsi agregasi (misal: menghitung total transaksi per pelanggan atau total barang terjual). Catat kompleksitas *query*-nya.

```sql
SELECT
    rj.id_ruas,
    rj.nama_ruas,
    SUM(k.jumlah_kendaraan) AS total_kendaraan
FROM
    ruas_jalan rj
JOIN
    kepadatan k ON rj.id_ruas = k.id_ruas
GROUP BY
    rj.id_ruas,
    rj.nama_ruas;
```

id_ruas	nama_ruas	total_kendaraan
1	Jl. Serayu	770
2	Jl. Salak	90

Pencarian Relasi (*Join*): Karena kolom penghubungnya (id_ruas) sudah dipasangi *Index* (seperti daftar isi di buku), database tidak perlu memeriksa satu per satu dari awal. Pencariannya sangat cepat menggunakan metode pohon terstruktur, rata-rata kecepatannya adalah $O(\log N)$.
Pengelompokan & Penjumlahan (*Group By*): Setelah datanya ketemu dan digabung, database perlu menyortir serta mengelompokkan baris-baris tersebut berdasarkan nama jalannya. Proses merapikan dan menghitung nilainya ini memakan waktu $O(N \log N)$ (artinya makin banyak data, waktu prosesnya akan bertambah secara bertahap).
Kebutuhan Memori: Database butuh "meja kerja sementara" (biasanya disebut *sort buffer* atau tabel sementara di RAM) untuk menampung hasil gabungan tabel dan menumpuk hasil penjumlahan sementara (SUM) sebelum hasilnya dimunculkan di layar. Besaran RAM yang dipakai ini sebanding dengan seberapa banyak jumlah baris data yang sedang diproses ($O(N)$).

2. Penerapan Denormalisasi (Tahap 2): Eksekusi DDL (ALTER TABLE atau CREATE TABLE) untuk menambahkan satu atribut turunan pada tabel master ATAU membuat satu tabel rekapitulasi yang menyimpan hasil dari perhitungan tahap 1.

	id_ruas	nama_ruas	jenis_jalan	lebar_jalan	panjang	kecepatan_normal	geometri	total_kendaraan
Edit Copy Delete	1	Jl. Serayu	Kolektor	10.00	2.50	40.00	LINESTRING(...)	770
Edit Copy Delete	2	Jl. Salak	Lokal	6.00	1.20	30.00	LINESTRING(...)	90

3. Pembuktian (Tahap 3): Tulis *query* SELECT sederhana untuk memanggil data dari kolom/tabel denormalisasi (Tahap 2) tanpa menggunakan JOIN atau agregasi. Bandingkan dengan *query* Tahap 1.

```sql
-- Query Tahap 3: Memanggil data tanpa JOIN / Agregasi langsung dari tabel ruas_jalan
SELECT
    id_ruas,
    nama_ruas,
    total_kendaraan
FROM
    ruas_jalan;
```

Perbandingan dengan query 1 dan 3 :

- Query Pertama (Atas): Cara lama yang lebih lambat. Setiap kali butuh data total kendaraan, sistem harus mengecek dan menggabungkan tabel ruas_jalan dengan tabel kepadatan secara *real-time* menggunakan JOIN dan SUM.
- Query Kedua (Bawah): Cara cepat dan optimal yang sudah diterapkan di Tahap 3. Karena kolom total_kendaraan sudah dibuat dan disimpan langsung di dalam tabel ruas_jalan, pemanggilan data bisa dilakukan secara instan tanpa perlu perintah JOIN lagi.

.

POST TES

1. Lampirkan kedua *script* SQL kelompok Anda: (1) *Query* JOIN yang kompleks, dan (2) *Script* DDL pembentukan kolom/tabel denormalisasi beserta *query* panggilannya!

```sql
SELECT
    rj.id_ruas,
    rj.nama_ruas,
    SUM(k.jumlah_kendaraan) AS total_kendaraan
FROM
    ruas_jalan rj
JOIN
    kepadatan k ON rj.id_ruas = k.id_ruas
GROUP BY
    rj.id_ruas,
    rj.nama_ruas;

    -- 1. Menambahkan kolom turunan ke tabel ruas_jalan
ALTER TABLE ruas_jalan
ADD COLUMN total_kendaraan INT DEFAULT 0;

-- 2. Mengisi nilai awal ke kolom denormalisasi berdasarkan hasil perhitungan sebelumnya
UPDATE ruas_jalan rj
SET total_kendaraan = (
    SELECT COALESCE(SUM(k.jumlah_kendaraan), 0)
    FROM kepadatan k
    WHERE k.id_ruas = rj.id_ruas
);
```

2. Berdasarkan percobaan kelompok Anda, jelaskan dengan bahasa sendiri kelebihan denormalisasi dalam memangkas penggunaan klausa JOIN pada sistem informasi!

Kelebihan utama denormalisasi adalah kita tidak perlu lagi menggabungkan tabel (*JOIN*) atau menghitung ulang data secara manual setiap kali *query* dijalankan. Hasil kalkulasi sudah disimpan

langsung di dalam satu kolom khusus, sehingga pembacaan data menjadi jauh lebih cepat, ringan, dan efisien bagi system

3. Jelaskan secara singkat, apa risiko utama terhadap data (contoh: inkonsistensi) yang muncul akibat kelompok Anda menambahkan atribut turunan atau tabel rekapitulasi tersebut?

Risiko utamanya adalah inkonsistensi data (data tidak sinkron).

Karena nilai totalnya disimpan secara permanen di dalam kolom tambahan, ketika ada penambahan, perubahan, atau penghapusan data transaksi baru (seperti data kepadatan kendaraan), nilai pada kolom turunan tersebut tidak akan otomatis berubah sendiri kecuali sistem melakukan pembaruan (update) ulang atau menggunakan *trigger*. Akibatnya, data yang tampil di laporan bisa berbeda atau tidak sesuai dengan kondisi transaksi aslinya.